Testing

Cohort 3 Team 1: Pixels of Promise

Movitz Aaron, Jake Adams, Oliver Belam, Anna Burt, Emily Foran, Sky Haines-Bass, Job Young

University of York

ENG1: Software & Systems Engineering

January 13, 2025

a.)
We used LibGDX's headless backend to streamline and improve the efficiency of our tests (implemented using JUnit). This allowed us to test the functionality of our code separate from the graphics rendering, thus drastically reducing testing runtime and enabling tests to be run via GitHub actions. This enabled us to integrate the tests into the developmental pipeline to check code functionality before releasing updates, improving the maintainability of the software.

As for our testing approaches, we aimed to increase our code coverage (measured using Jacoco) as much as reasonably possible, whilst holding the testing of more complex methods and scenarios as a higher priority. This approach enabled us to consider what levels of structural and statement coverage would be feasible compared to what would be appropriate. This was appropriate for the project as, when considering code coverage levels to aim for, we also factored in the project scale, deadlines and product brief. This allowed us to work towards realistic implementation goals, boosting team morale and ensuring upheld clarity regarding the direction we wanted to take the project in.

When developing tests, we aimed to consider both black box and white box approaches. For example, in tests 3 and 9, we tested that the constructor values and textures were instantiated correctly, based on the values specified in the script, clearly demonstrating a white box testing approach. On the other hand, we also completed some manual tests for the UI elements, which assessed the system's visual responses to various specified user stimuli. These tests are an explicit example of the black box testing approach. By combining both the white and black box testing methods, we were able to benefit from the robustness obtained when considering each individual code branch (white box testing), whilst also evaluating the functionality of the system and assessing its fulfilment of the specified requirements (black box testing).

When picking inputs, we aimed to utilise both random sampling and input space partitioning to help minimise designer bias, whilst still assessing most equivalent test classes. We also aimed to maintain a small size and narrow scope for each test produced, allowing us to minimise the time taken to run and, if necessary, debug upon a test failure. To expand on this, we also implemented some boundary testing, for example, in test 2 where we try to place a building on the edge of the map and implicitly in the manual tests regarding building placement, as any noted incorrect placements would be recorded. This allowed us to ensure that the system functioned as intended regardless of user input, improving the robustness of the software.

In situations where automated testing was not feasible or perhaps manual testing was heavily preferred, we created a manual testing plan detailing: testing properties, why the test was required, when the test was to be carried out and who would execute the test and analyse the results. This enabled us to carry out rigorous manual testing, without sacrificing the documentability of the tests and their outcomes.

b) **Test Report Overview**
Within the current implementation of our game (version v0.4.0), our automated test suite reports that 100% of our tests (all 73 tests across 15 classes) pass successfully. The Jacoco report generated via all GitHub actions (and specifically the report for v0.4.0) confirms complete success with no failed tests. Additionally, runtime statistics on GitHub Actions for a build job (this includes testing, test coverage and a checkstyle) indicate an average total runtime of approximately 1 minute for the build, which demonstrates not only the efficiency of our test suite but also the scalability of the system - many further tests could be added without significantly increasing or impacting the runtime.

Test Completeness and Coverage
Most of our tests are unit tests, testing that a class functions as it should, or is expected to, testing individual and small parts of a class to eventually cover the complete class. However, we did not use a lot of integration testing, there are some tests such as `AchievementManagerTest` which uses the `gameState` class in its tests to ensure that the `AchievementManager` reads values from said `gameState` correctly and does the right thing as a result of this.
To evaluate test completeness, we refer to Jacoco's coverage reports (found on website). In the most recent version, v0.4.0, the overall coverage is reported as 23% (covering 2,204 of 9,541 instructions across 118 of the 444 possible code branches). However, this overall figure includes UI and associated Gdx files and scripts that were unfortunately not fully automated, skewing the results.
Moreover, key areas relevant to our automated unit and integration tests show much higher coverage:

- Building package: 97% (the 3% we're missing here is two lines in the `Building` class which tries to load a Texture using Gdx, which was very hard to mock or implement, and therefore we omitted)
- Scenario package: 41% (here we are testing everything that we could, the problem with a scenario for us was the `Timer`, there are few ways of testing if the timer has ran out in the correctly allocated time due to problems with Actions and the Linux machine that the jobs were running on, sometimes the tests would fail, and sometimes the tests would pass. This was despite having the code wait the correct amount of time and rounding quite heavily, so we decided to omit this from the testing and instead test the timer manually, starting a stopwatch and waiting to see if the timing was correct (or near enough))
- Tile package: 75% (again, similarly to the building package, we manage to test everything apart from loading a Texture, so in reality our test coverage for this set of classes is nearing 100%)
- Pathway package: 47% (here we are at the same problem as before, textures. Without them we cover all possible lines of the pathways ensuring that they work correctly, testing them at their limits, their boundaries and everything in between)

These figures align with our goal of increasing meaningful coverage to approximately 50%. While the UI and some game elements were manually tested - e.g., visual evaluation of UI buttons, with `System.Out.println` calls to ensure the correct dialog choice - this manual testing complements our automated testing suite without reducing its significance.

UI Testing Challenges and Manual Testing Approach

As mentioned above, we did not test any class that utilised UI components, this is because of our headless setup, there is no display or actual rendering, limiting the ability to directly test behaviours or user interactions. We opted for more manual testing in UI components since UI elements are intended for human interaction, and thus we were able to directly observe UI behavior, layout and responsiveness. For example, when implementing Dialogs for the Scenarios, we found no effective ways of testing the button presses, so we added a `println` statement telling us which option the user had chosen and manually checked every option returned the correct result, and that this result would affect gameplay in the desired way.

By concentrating automated tests on non-UI code (such as game logic, event handling, etc.) using fakes and mocks, we ensured that the underlying logic is sound. Our manual testing of the UI would then verify that these components integrate correctly with the visual elements.

Readability, Consistency and Test Format

All tests follow a consistent and readable format adhering to Google style guidelines, verified by our CI's checkstyle reports. Although minor style violations are still noted in our reports (even in v0.4.0) - such as naming conventions that conflict with project-wide practices - these do not impact test functionality. For example, naming conversions like `UIElement` were preserved despite checkstyle suggestions to alter casing, to maintain consistency with our project.

Tests are structured into a familiar "set up, arrange, act, assert, and tear down" format:

- Setup/Arrange: Establish test data and environment
- Act: Execute functionality under test
- Assert: Verify the outcomes
- Tear down: Clean up and reset the environment, ready for the next test

This structure aids readability, debugging, and ensures test isolation - preventing side effects, such as race conditions, that could cause false positives or negatives. This was a big problem when trying to test `GameState`, as tests would run in parallel to speed up execution time whilst we had a singleton of `GameState`. This meant that, whilst one test was checking that the `money` variable was set to 0, another test was simultaneously setting that variable to 100. This resulted in inconsistencies in build/test successes, despite no alterations being made to the involved scripts, posing a challenge to debug. Due to this, in future, we will try to avoid using singletons.

Traceability and Relevance to Requirements

To monitor relevance and traceability between tests and requirements, a traceability matrix was created and maintained, which is available on the team website. This matrix maps each test to specific system requirements, using an "X" to denote coverage. This approach ensures:

- All requirements are addressed by at least one automated or manual test.
- Identification of redundant or unnecessary tests.
- Focus on improving coverage and eliminating gaps where crucial.

Although most requirements are covered, some (e.g. requirement 1.2) lack tests - this can be seen on our traceability matrix. This identifies areas for possible future test development.

Regression and Edge Case Testing

Our tests are designed to not only validate functionality, but to also serve as a regression suite for future changes. To clarify, this is a collection of test cases that are executed to ensure that recent code updates and/or changes have not broken existing functionality. By covering basic functionality and edge cases:

- We ensure that new commits do not break existing features.
- Regression testing forms the backbone of our CI approach, aligning with a testing pyramid methodology that balances unit (most heavily used), integration (somewhat used), and system tests (only done manually, by playing the game).
- Ensures focus on edge cases, which aids in identifying boundary errors and potential pitfalls that standard testing might overlook.

Examples of Testing Techniques Used

In `AchievemenetManagerTest` we utilise a fake - a stand-in object that mimics the behaviour of a real component but uses a simplified or in-memory implementation - to simulate persistent storage operations without needing actual file I/O or external resources. By doing this, we achieved a higher test isolation, sped up tests by avoiding real disk access and simplified our setup and tear down operations.

In our test, we fake the `Preferences` class, using a new class called `FakePreferences`. This fake supports and implements only the subset of methods required for the tests, throwing `UnsupportedOperationException` for unused methods; uses a simple `HashMap` internally to mimic persistent key-value storage, demonstrating a light way to simulate complex behaviour.

Additionally, in the same test class we use static mocking to enable control over, what would otherwise be, rigid or hard-to-test static methods, in turn leading to more predictable and isolated tests. In several tests (e.g. `testPollUnlockBroke`), we use Mockito's static mocking feature (`MockedStatic`) to intercept static calls to `EventHandler.getEventHandler()`. This approach prevents actual static calls from executing, which would lead to unwanted dependency on the EventHandler (which is set up with all of its Events in run-time, therefore making it very difficult to be able to call these methods and test if they were called in a headless testing environment). It also allows verification that specific events are triggered appropriately, without invoking the real event-handling logic. On the other hand, we can also verify that some events were never triggered, which can be very useful. For example, in all of our tests which unlock achievements, first we verify that the achievement is not unlocked and that the EventManager has never called the achievement specific method, then we meet the requirements of the achievement, poll the manager and check that the method has been called. This helps the system verify the correct triggers of events, stopping unintended behaviour early by catching it, hence upholding test correctness.

Static mocking was necessary due to reliance on static methods in the `EventManager` that were hard to override or inject normally. Faking was used to speed up our tests, to enable easier debugging and ensuring better isolation. However, there were difficulties trying to match the fake to the real implementation (there could be a test testing that the fake is the same as the real) but they were overcome with time and patience.